

Designing Your User Interface with Views

Dr. Shobhit Tyagi

Assistant Professor

Department of Computer Science and Engineering & Information Technology

Jaypee Institute of Information Technology (JIIT), Noida, Uttar Pradesh, India

TOPICS:

1. Designing Your User Interface with Views

Using Basic Views

- TextView
- Button, ImageButton, EditText, CheckBox
- ToggleButton, RadioButton, and RadioGroup Views
- ProgressBar View
- AutoCompleteTextView View

2. Using Picker Views

- TimePicker View
- DatePicker View

3. Using List Views to Display Long Lists

- ListView View
- Using the Spinner View

4. Understanding Specialized Fragments

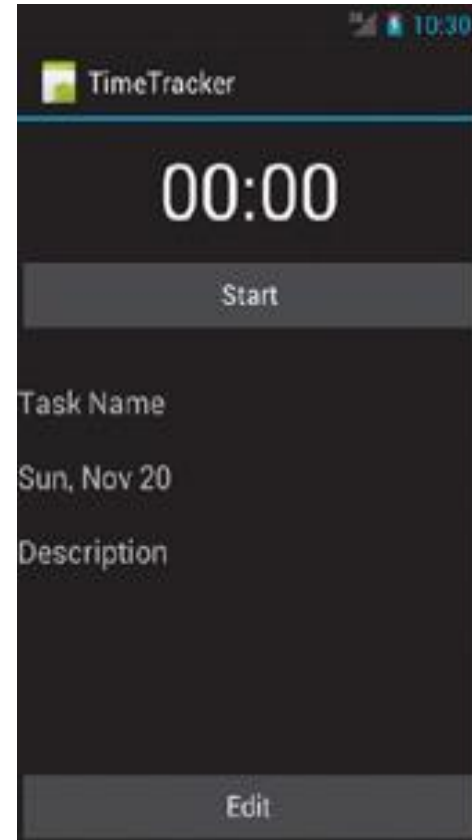
- Using a ListFragment
- Using a DialogFragment
- Using a PreferenceFragment

Introduction

- The most basic element of Android user interfaces is the View class. A view represents an area of the screen.
- Buttons, lists, webpages, and even empty spaces are represented by views.
- Android contains a rich array of pre-built View classes that provide much of the functionality you will need.
- When the built-in views aren't enough, it's possible to create special views that are just right for your application.

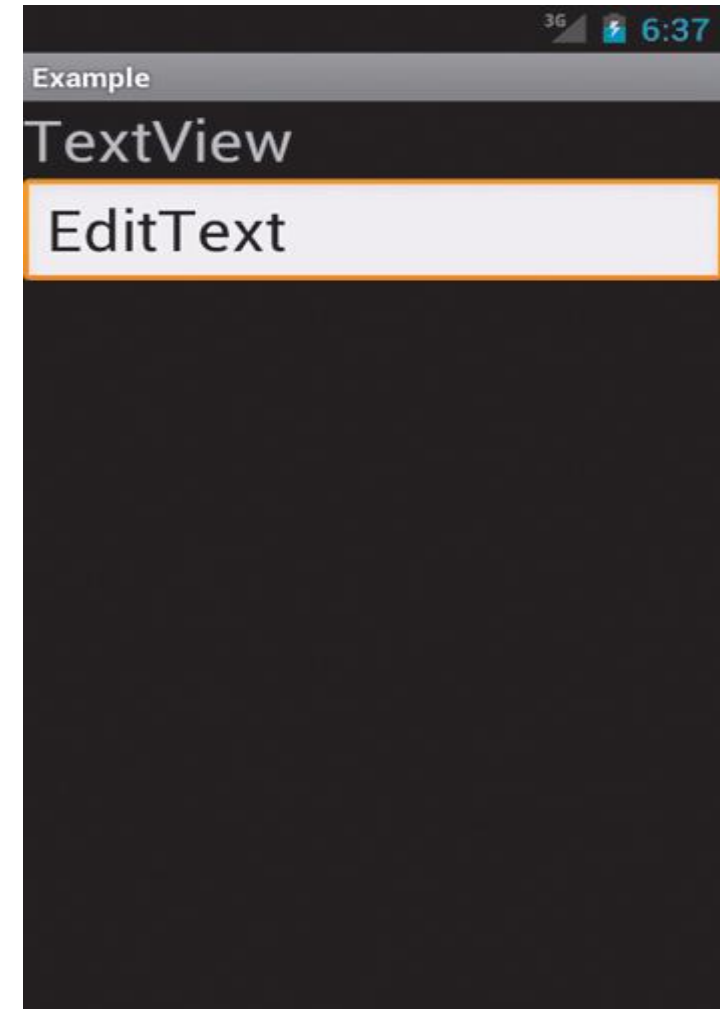
Creating a Basic Form

- The TimeTracker app looks pretty good so far, but it's time to add more than just a list of times, you'll add some text entry forms and split the app into multiple activities.
- When you're finished, you'll have something that looks like figure This section will cover the basic widgets you see in the image, as well as how to arrange them.



TextView and EditText

- The most basic view available on Android is the TextView, which allocates an area of the screen to display text.
- You will use this view a lot in your layouts.
- An EditText is a TextView that is configured to allow the user to edit the text inside it .
- Tapping an EditText will display a cursor and the device software keyboard, allowing the user to enter new text or edit the existing text.
- The TextView has optional attributes such as size, font, and color that allow you to change the appearance of the text

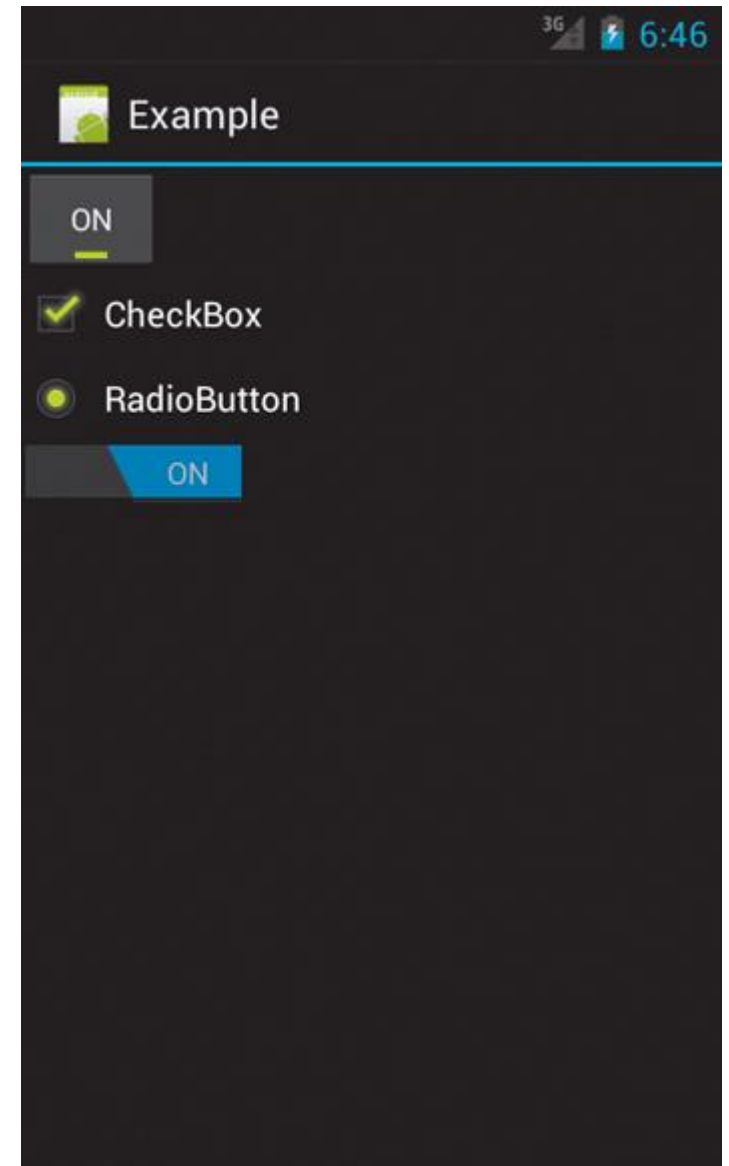


Buttons

- Buttons are simply TextViews that have a special background image—this background is actually an XML file that lists the images that should be used for the different button states (normal, hovered, focused, and pressed). This type of XML resource is called a state list resource
- ```
<Button android:id="@+id/finished"
 android:layout_width="match_parent"
 android:layout_height="wrap_content" android:text="@string/finished" >
</Button>
```
- ```
<Button android:id="@+id/edit" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:text="@string/edit" >
</Button>
```

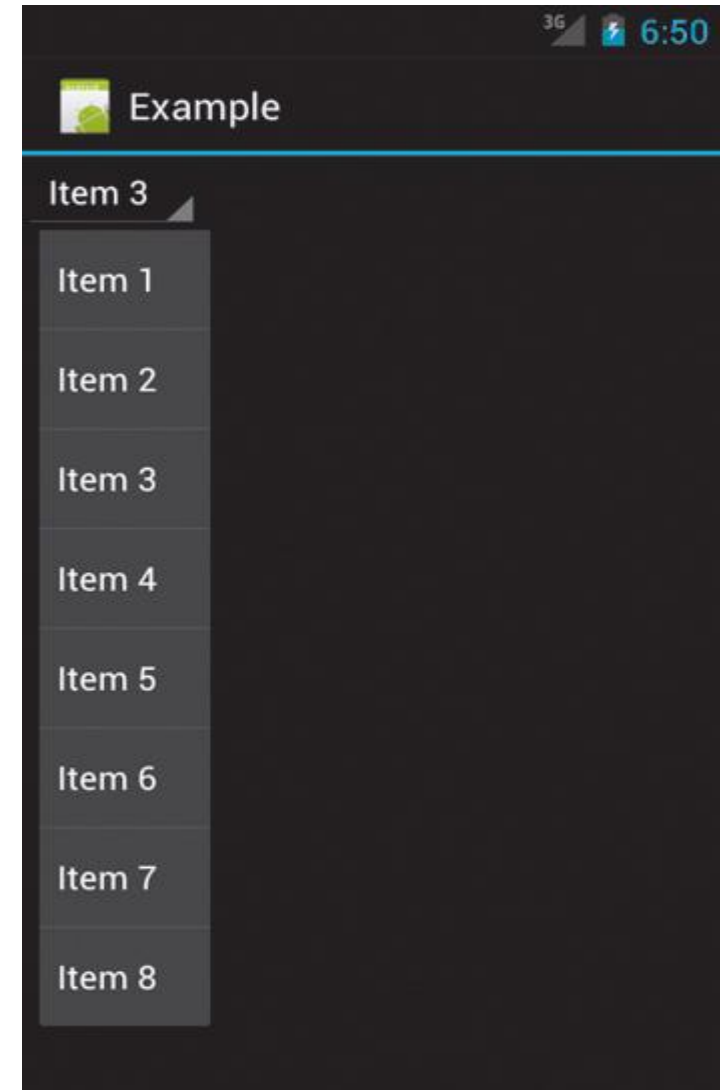
Boolean Buttons

- Buttons are convenient for indicating on/off states.
- Android has a number of views, including toggle buttons, checkboxes, and radio buttons, that subclass the Button class and present a toggle between a true value and a false value.
- In addition, Android 4.0 introduced an option called the switch.
- Figure shows all these options for the 4.0 release of Android.



Spinners

- A spinner looks like a button and displays a list of choices when pressed.
- Figure shows an example of a spinner choice list.
- The options presented by a spinner can be specified using the XML `android:entries` attribute



ScrollView

- Adding entry fields to a form is simple, but what happens if you cannot fit all the views on one screen?
- In these cases, it's often useful to allow scrolling in order to fit more elements in a single activity.
- To achieve this effect, you need to wrap your views in a ScrollView container.
- A ScrollView allows you to create a view that is larger than the physical screen on a device and scroll it to reveal the full contents.
- ScrollView is actually a subclass of FrameLayout, but it adds the ability to scroll its content.
- You typically place another layout container inside the ScrollView to arrange the child views.

```
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true" >
    <LinearLayout>
        <!-- Rest of code here -->
    </LinearLayout>
</ScrollView>
```

ImageButton

- Displays a button with an image (instead of text) that can be pressed or clicked by the user.
- By default, an ImageButton looks like a regular Button, with the standard button background that changes color during different button states.
- The image on the surface of the button is defined either by the `android:src` attribute in the `<ImageButton>` XML element or by the `ImageView.setImageResource(int)` method.
- To remove the standard button background image, define your own background image or set the background color to be transparent.
- To indicate the different button states (focused, selected, etc.), you can define a different image for each state.
- E.g., a blue image by default, an orange one for when focused, and a yellow one for when pressed. An easy way to do this is with an XML drawable "selector." For example:

ImageButton

```
<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/res/android">
  <item android:state_pressed="true"
        android:drawable="@drawable/button_pressed" /> <!-- pressed -->
  <item android:state_focused="true"
        android:drawable="@drawable/button_focused" /> <!-- focused -->
  <item android:drawable="@drawable/button_normal" /> <!-- default -->
</selector>
```

EditText

- A user interface element for entering and modifying text.
- When you define an edit text widget, you must specify the `R.styleable.TextView_inputType` attribute.
- For example, for plain text input set `inputType` to "text":

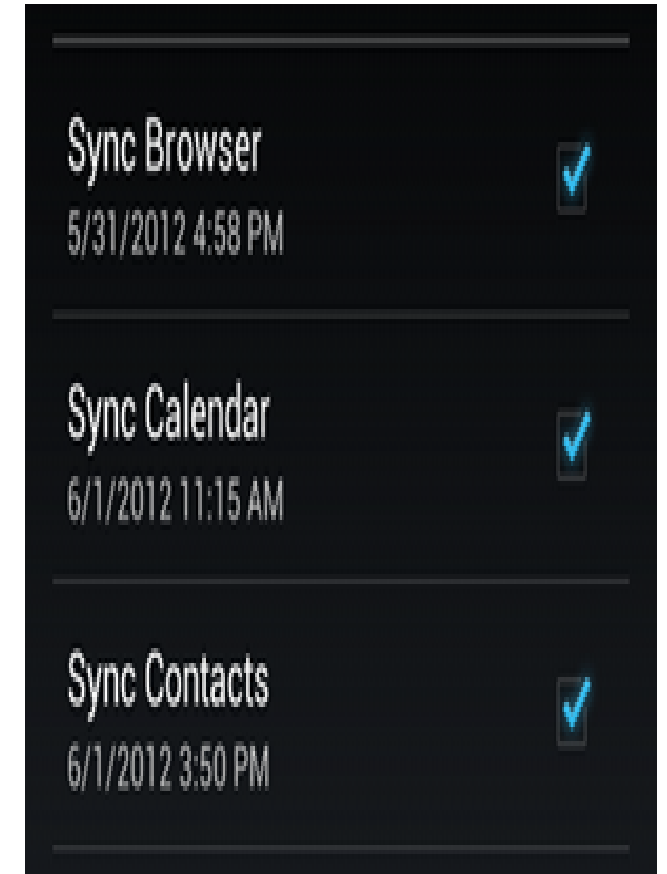
```
<EditText  
    android:id="@+id/plain_text_input"  
    android:layout_height="wrap_content"  
    android:layout_width="match_parent"  
    android:inputType="text"/>
```

EditText

- Choosing the input type configures the keyboard type that is shown, acceptable characters, and appearance of the edit text.
- For example, if you want to accept a secret number, like a unique pin or serial number, you can set `inputType` to `"numericPassword"`.
- An `inputType` of `"numericPassword"` results in an edit text that accepts numbers only, shows a numeric keyboard when focused, and masks the text that is entered for privacy.

Checkboxes

- Checkboxes allow the user to select one or more options from a set. Typically, you should present each checkbox option in a vertical list
- To create each checkbox option, create a `CheckBox` in your layout. Because a set of checkbox options allows the user to select multiple items, each checkbox is managed separately and you must register a click listener for each one.



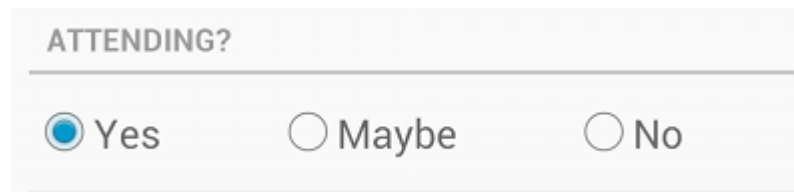
Toggle Buttons



- A toggle button allows the user to change a setting between two states.
- You can add a basic toggle button to your layout with the `ToggleButton` object.
- Android 4.0 (API level 14) introduces another kind of toggle button called a switch that provides a slider control, which you can add with a `Switch` object.
- `SwitchCompat` is a version of the `Switch` widget which runs on devices back to API 7.
- If you need to change a button's state yourself, you can use the `CompoundButton.setChecked()` or `CompoundButton.toggle()` method.

Radio Buttons

- Radio buttons allow the user to select one option from a set.
- You should use radio buttons for optional sets that are mutually exclusive if you think that the user needs to see all available options side-by-side.
- If it's not necessary to show all options side-by-side, use a spinner instead.



ATTENDING?

☒ Yes ☐ Maybe ☐ No

- To create each radio button option, create a `RadioButton` in your layout.
- However, because radio buttons are mutually exclusive, you must group them together inside a `RadioGroup`.
- By grouping them together, the system ensures that only one radio button can be selected at a time.

RadioGroup Views

- A RadioGroup class is used for set of radio buttons.
- If we check one radio button that belongs to a radio group, it automatically unchecks any previously checked radio button within the same group
- RadioGroup Attributes
- android:checkedButton - This is the id of child radio button that should be checked by default within this radio grou

RadioGroup Views

- `android:background`
 - This is a drawable to use as the background.
- `android:contentDescription`
 - This defines text that briefly describes content of the view.
- `android:id`
 - This supplies an identifier name for this view
- `android:onClick`
 - This is the name of the method in this View's context to invoke when the view
- `android:visibility`
 - This controls the initial visibility of the view.is clicked.

ProgressBar View

- Android ProgressBar is a graphical view indicator that shows some progress.
- Android progress bar displays a bar representing the completing of the task.
- Progress bar in android is useful since it gives the user an idea of time to finish its task.

Android ProgressBar attributes

- **android:max :**
 - We can set the maximum value of the ProgressBar using this attribute. By default the progress bar maximum value is 100
- **android:indeterminate :**
 - A boolean value is set depending on whether the time is determinate or not. Setting this attribute to false would show the actual progress. Else if it's set to true a cyclic animation is displayed to show that progress is happening
- **android:minHeight :**
 - It's used to set the height of the ProgressBar
- **android:minWidth :**
 - It's used to set the width of the ProgressBar
- **android:progress :**
 - It's used to set the number by which the progress bar value will be incremented
- **style :**
 - By default the progress bar will be displayed as a spinning wheel. If we want it to be displayed as a horizontal bar, we need to set the attribute as :
style="?android:attr/progressBarStyleHorizontal"

Pickers

- Android provides controls for the user to pick a time or pick a date as ready-to-use dialogs. Each picker provides controls for selecting each part of the time (hour, minute, AM/PM) or date (month, day, year).
- Using these pickers helps ensure that your users can pick a time or date that is valid, formatted correctly, and adjusted to the user's locale.

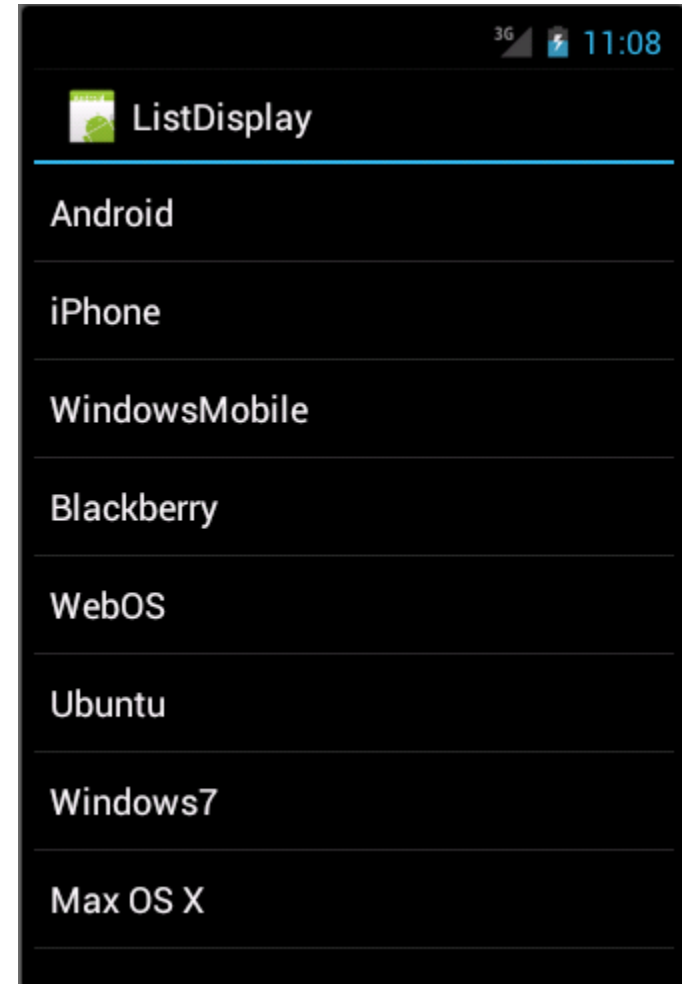


Pickers

- DialogFragment to host each time or date picker.
- The DialogFragment manages the dialog lifecycle for you and allows you to display the pickers in different layout configurations, such as in a basic dialog on handsets or as an embedded part of the layout on large screens.

Using List Views to Display Long Lists

- Android **ListView** is a view which groups several items and display them in vertical scrollable list.
- The list items are automatically inserted to the list using an **Adapter** that pulls content from a source such as an array or database.



Using List Views to Display Long Lists

- An adapter actually bridges between UI components and the data source that fill data into UI Component. Adapter holds the data and send the data to adapter view, the view can takes the data from adapter view and shows the data on different views like as spinner, list view, grid view etc.
- The ListView and GridView are subclasses of AdapterView and they can be populated by binding them to an Adapter, which retrieves data from an external source and creates a View that represents each data entry.
- Android provides several subclasses of Adapter that are useful for retrieving different kinds of data and building views for an AdapterView (i.e. ListView or GridView). The common adapters are ArrayAdapter,Base Adapter, CursorAdapter, SimpleCursorAdapter,SpinnerAdapter and WrapperListAdapter. We will see separate examples for both the adapters

ListView Attributes

- **android:id**
 - This is the ID which uniquely identifies the layout.
- **android:divider**
 - This is drawable or color to draw between list items.
- **android:dividerHeight**
 - This specifies height of the divider. This could be in px, dp, sp, in, or mm.
- **android:entries**
 - Specifies the reference to an array resource that will populate the ListView.
- **android:footerDividersEnabled**
 - When set to false, the ListView will not draw the divider before each footer view. The default value is true.
- **android:headerDividersEnabled**
 - When set to false, the ListView will not draw the divider after each header view. The default value is true.

Using a RecyclerView to display list items

- The RecyclerView is a component and library of the Android SDK that makes it much simpler to display long lists. Before the RecyclerView came along, displaying a list with a large or unknown amount of items would be an involved process.
- **The Item Layout**
 - The first step to using a RecyclerView would typically be to design the layout that will be used for each item.
 - The layout below contains two TextView views positioned one above the other.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="72dp">

    <TextView
        android:id="@+id/title_text_view"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="14dp"
        app:layout_constraintBottom_toTopOf="@id/subtitle_text_view"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent"
        tools:text="Example Title Text" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

The Item

- The list will need a list of items to display.

```
data class OrderItem  
(  
    val itemId: String,  
    val displayName: String  
)
```

Adapter Components

- The adapter has one construction parameter: The list of items to be displayed.
- There is a nested class called `OrderViewHolder`. This represents the reusable view of a single list item and should extend `RecyclerView.ViewHolder`.
- `getItemCount` should return the number of items in our list.
- `onCreateViewHolder` should inflate the XML layout of our list item we created earlier and create an `OrderViewHolder` from it.
- `onBindViewHolder` should update the view holder with the item at the provided position.
- The adapter should extend `RecyclerView.Adapter`, using your `ViewHolder` class as the type argument.

Android Spinner

- Android Spinner is just a drop down list similar to what's seen in other programming languages such as in HTML pages.
- In Android, Spinner is used to select one value from a set of values. In the default state, a spinner shows its currently selected value. Touching the spinner displays a drop down menu with all other available values, from which the user can select a new one.
- Android spinner is associated with AdapterView. So we need to set the adapter class with the Spinner.
- **Spinner spinner = (Spinner) findViewById(R.id.spinner);**

Using a ListFragment

- A fragment that displays a list of items by binding to a data source such as an array or Cursor, and exposes event handlers when the user selects an item.
- ListFragment hosts a ListView object that can be bound to different data sources, typically either an array or a Cursor holding query results
- public class ListFragment
- extends Fragment
- java.lang.Object
 - ↳ android.app.Fragment
 - ↳ android.app.ListFragment

Screen Layout

- ListFragment has a default layout that consists of a single list view. However, if you desire, you can customize the fragment layout by returning your own view hierarchy from onCreateView(LayoutInflater, ViewGroup, Bundle).
- To do this, your view hierarchy must contain a ListView object with the id "`@android:id/list`" (or `R.id.list` if it's in code)

Using a DialogFragment

- A fragment that displays a dialog window, floating on top of its activity's window.
- This fragment contains a Dialog object, which it displays as appropriate based on the fragment's state.
- Control of the dialog (deciding when to show, hide, dismiss it) should be done through the API here, not with direct calls on the dialog.
- `Fragment.onCreateView(android.view.LayoutInflater, android.view.ViewGroup, android.os.Bundle)` to supply the content of the dialog.
- Alternatively, they can override `onCreateDialog(android.os.Bundle)` to create an entirely custom dialog, such as an AlertDialog, with its own content.

Basic Dialog

- The simplest use of DialogFragment is as a floating container for the fravoid
showDialog() {
 - mStackLevel++;
 - // DialogFragment.show() will take care of adding the fragment
 - // in a transaction. We also want to remove any currently showing
 - // dialog, so make our own transaction and take care of that here.
 - FragmentTransaction ft = getFragmentManager().beginTransaction();
 - Fragment prev = getFragmentManager().findFragmentByTag("dialog");
 - if (prev != null) {
 - ft.remove(prev);
 - }
 - ft.addToBackStack(null);
 - // Create and show the dialog.
 - DialogFragment newFragment = MyDialogFragment.newInstance(mStackLevel);
 - newFragment.show(ft, "dialog");
 - }fragment's view hierarchy

Alert Dialog

- Instead of implementing `Fragment.onCreateView(LayoutInflater, ViewGroup, Bundle)` to generate the view hierarchy inside of a dialog, you may implement `onCreateDialog(android.os.Bundle)` to create your own custom Dialog object.
- AlertDialog, allowing you to display standard alerts to the user that are managed by a fragment.

```

public static class MyAlertDialogFragment extends DialogFragment {

    public static MyAlertDialogFragment newInstance(int title) {
        MyAlertDialogFragment frag = new MyAlertDialogFragment();
        Bundle args = new Bundle();
        args.putInt("title", title);
        frag.setArguments(args);
        return frag;
    }

    @Override
    public Dialog onCreateDialog(Bundle savedInstanceState) {
        int title = getArguments().getInt("title");

        return new AlertDialog.Builder(getActivity())
            .setIcon(R.drawable.alert_dialog_icon)
            .setTitle(title)
            .setPositiveButton(R.string.alert_dialog_ok,
                new DialogInterface.OnClickListener() {
                    public void onClick(DialogInterface dialog, int whichButton) {
                        ((FragmentAlertDialog)getActivity()).doPositiveClick();
                    }
                }
            )
            .setNegativeButton(R.string.alert_dialog_cancel,
                new DialogInterface.OnClickListener() {
                    public void onClick(DialogInterface dialog, int whichButton) {
                        ((FragmentAlertDialog)getActivity()).doNegativeClick();
                    }
                }
            )
            .create();
    }
}

```

Using a PreferenceFragment

- public abstract class PreferenceFragment
- extends Fragment implements
PreferenceManager.OnPreferenceTreeClickListener,
PreferenceManager.OnDisplayPreferenceDialogListener,
PreferenceManager.OnNavigateToScreenListener,
DialogPreference.TargetFragment
- java.lang.Object
- ↳ android.app.Fragment
- ↳ androidx.preference.PreferenceFragment

Using a PreferenceFragment

- Shows a hierarchy of Preference objects as lists. These preferences will automatically save to **SharedPreferences** as the user interacts with them. To retrieve an instance of **SharedPreferences** that the preference hierarchy in this fragment will use, call **PreferenceManager.getDefaultSharedPreferences(android.content.Context)** with a context in the same package as this fragment.
- A **PreferenceScreen** object should be at the top of the preference hierarchy. Furthermore, subsequent **PreferenceScreen** in the hierarchy denote a screen break--that is the preferences contained within subsequent **PreferenceScreen** should be shown on another screen.
- The preference framework handles this by calling **onNavigateToScreen(PreferenceScreen)**.

Using a PreferenceFragment

- The preference hierarchy can be formed in multiple ways:
 - From an XML file specifying the hierarchy
 - From different Activities that each specify its own preferences in an XML file via Activity meta-data
 - From an object hierarchy rooted with PreferenceScreen
 - To inflate from XML, use the `addPreferencesFromResource(int)`. The root element should be a `PreferenceScreen`. Subsequent elements can point to actual Preference subclasses. As mentioned above, subsequent `PreferenceScreen` in the hierarchy will result in the screen break.

Working with the AppBar

- When using fragments, the app bar can be implemented as an ActionBar that is owned by the host activity or a toolbar within your fragment's layout. Ownership of the app bar varies depending on the needs of your app.
- If all your screens use the same app bar that's always at the top and spans the width of the screen, then you should use a theme-provided action bar hosted by the activity. Using theme app bars helps to maintain a consistent look and provides a place to host option menus and an up button.



Working with the AppBar

- Use a toolbar hosted by the fragment if you want more control over the size, placement, and animation of the app bar across multiple screens. For example, you might need a collapsing app bar or one that spans only half the width of the screen and is vertically centered.
- Understanding the different approaches and employing a correct one saves you time and helps to ensure your app functions properly. Different situations require different approaches for things like inflating menus and responding to user interaction.

```
<!-- sample_menu.xml -->
<menu
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto">

  <item
    android:id="@+id/action_settings"
    android:icon="@drawable/ic_settings"
    android:title="@string/settings"
    app:showAsAction="ifRoom"/>
  <item
    android:id="@+id/action_done"
    android:icon="@drawable/ic_done"
    android:title="@string/done"
    app:showAsAction="ifRoom|withText"/>

</menu>
```

Activity-owned app bar

- The app bar is most commonly owned by the host activity. When the app bar is owned by an activity, fragments can interact with the app bar by overriding framework methods that are called during fragment creation

Register with activity

- You must inform the system that your app bar fragment is participating in the population of the options menu. To do this, call `setHasOptionsMenu(true)` in your fragment's `onCreate(Bundle)` method, as shown in the following example:

```
class ExampleFragment : Fragment() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setHasOptionsMenu(true)  
    }  
}
```

Register with activity

- `setHasOptionsMenu(true)` tells the system that your fragment would like to receive menu-related callbacks.
- When a menu-related event occurs (creation, clicks, and so on), the event handling method is first called on the activity before being called on the fragment.
- Note that your application logic should not depend on this order.
- If multiple fragments are hosted by the same activity, each fragment can supply menu options.
- In this case, the callback order depends on the order in which the fragments were added.

Inflating a menu

- To merge your menu into the app bar's options menu, override `onCreateOptionsMenu()` in your fragment. This method receives the current app bar menu and a `MenuInflater` as parameters. Use the menu inflater to create an instance of your fragment's menu, and then merge it into the current menu, as shown in the following example:

```
class ExampleFragment : Fragment() {  
    ...  
  
    override fun onCreateOptionsMenu(menu: Menu, inflater:  
MenuInflater) {  
        inflater.inflate(R.menu.sample_menu, menu)  
    }  
}
```



Handling click events

- Every activity and fragment that participates in the options menu is able to respond to touches. Fragment's `onOptionsItemSelected()` receives the selected menu item as a parameter and returns a boolean to indicate whether or not the touch has been consumed. Once an activity or fragment returns true from `onOptionsItemSelected()`, no other participating fragments will receive the callback.
- In your implementation of `onOptionsItemSelected()`, use a switch statement on the `itemId` of the menu item. If the selected item is yours, handle the touch appropriately and return true to indicate that the click event has been handled. If the selected item is not yours, call the super implementation. By default, the super implementation returns false to allow menu processing to continue.

```
class ExampleFragment : Fragment() {  
    ...  
  
    override fun onOptionsItemSelected(item: MenuItem): Boolean {  
        return when (item.itemId) {  
            R.id.action_settings -> {  
                // navigate to settings screen  
                true  
            }  
            R.id.action_done -> {  
                // save profile changes  
                true  
            }  
            else -> super.onOptionsItemSelected(item)  
        }  
    }  
}
```

THANK YOU

ANY QUESTIONS?

